

MCP (Model Context Protocol).

* This is an open standard that provides a universal interface for AI applications to connect with external tools, data sources & systems - securely & consistently.

→ Launched by Anthropic.

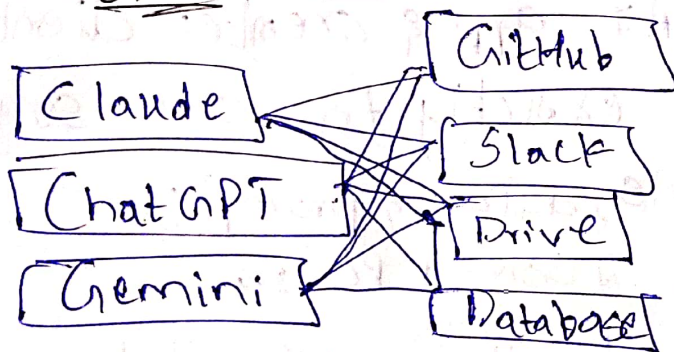
→ Supported across all the ecosystems.

Ex:- OpenAI, Oracle, Microsoft, AWS.

→ JSON-RPC 2.0 is the opensource.

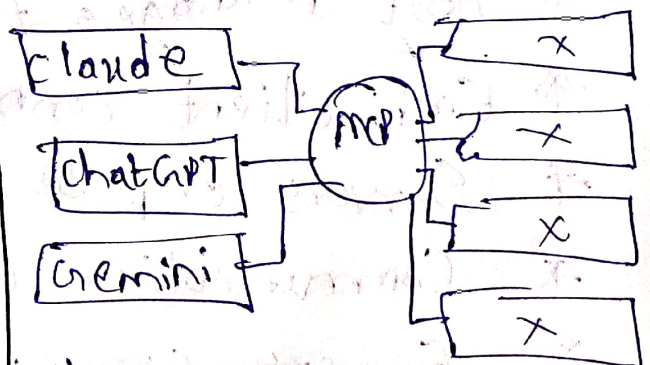
Remote [✓] Procedure Call.

Before MCP



Here, every AI app. needs to connect with the tool. So $N \times M$
 $3 \times 4 = 12$
integrations.

After MCP

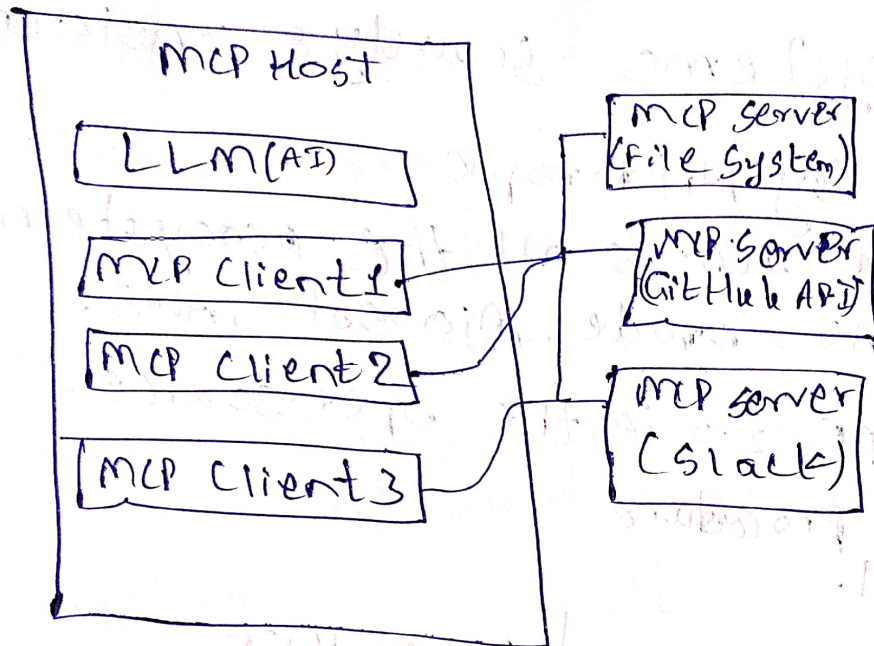


Here $N + M = 3 + 4$
 $= 7$
integrations
* This reduces the complexity.

MCP Architecture

* This contains 3 components.

1) MCP Host



- * Host manages the AI app & creates clients.
- * Each client connects exactly to one server.
- * Servers expose tools, data & prompts.
- * Communication uses JSON-RPC 2.0.
- * Servers can run locally or remotely.

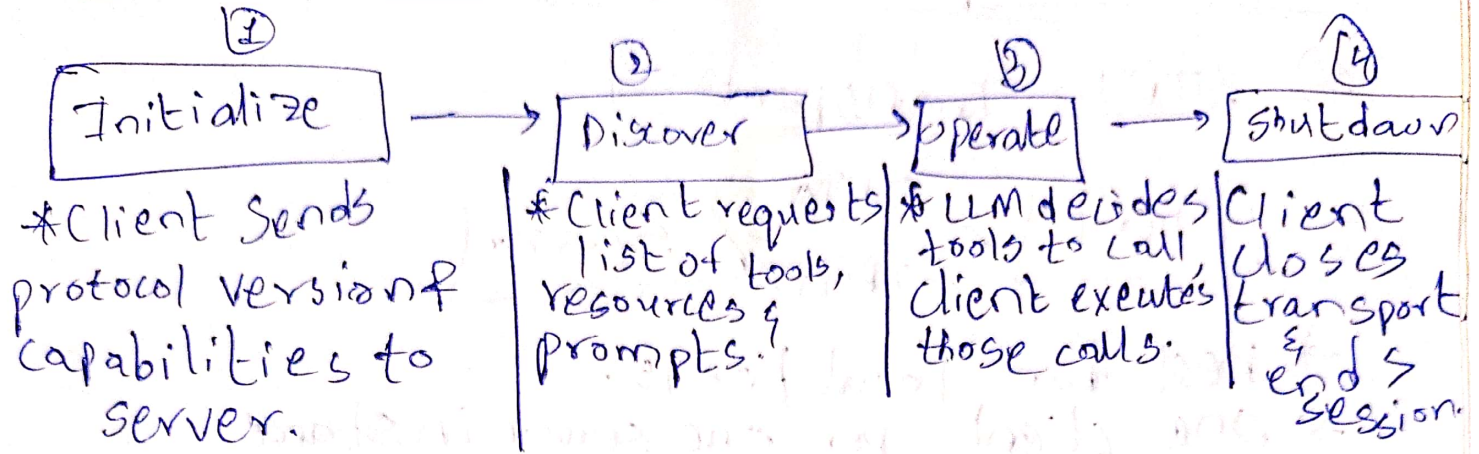
Core Primitives

Tools ⇒ Functions the AI can call to perform actions.
(Model Controlled). * Tools are commonly used primitive.

Resources ⇒ Structured data the AI can read for context.
(Application - controlled).

Prompts ⇒ Templates that structure LLM interactions.
(user-controlled).

MCP Connection Lifecycle



JSON-RPC 2.0

* Used for bi-directional which is better than REST API.

JSON-RPC messages include,

→ jsonrpc (always "2.0")

→ id (used to match requests & responses).

* FastMCP & Lang Chain are used to write these messages.

MCP Method 1 ⇒ tools/list

* This is called when the client connects to discover available tools, & server returns every tool it offers.

* Here Agent discovers tools without user hardcoding them.

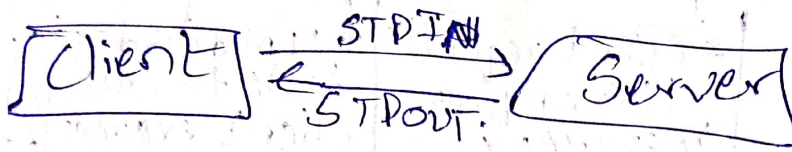
MCP Method 2 ⇒ tools/call

* This is called each time the LLM decides it needs a tool.

* The LLM decides to call this tool based on the user's request & available tool descriptions.

MCP Transport Mechanisms

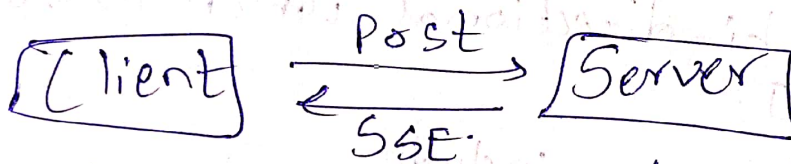
1) STDIO Transport



* Best for local tools.

* One client per one server instance.

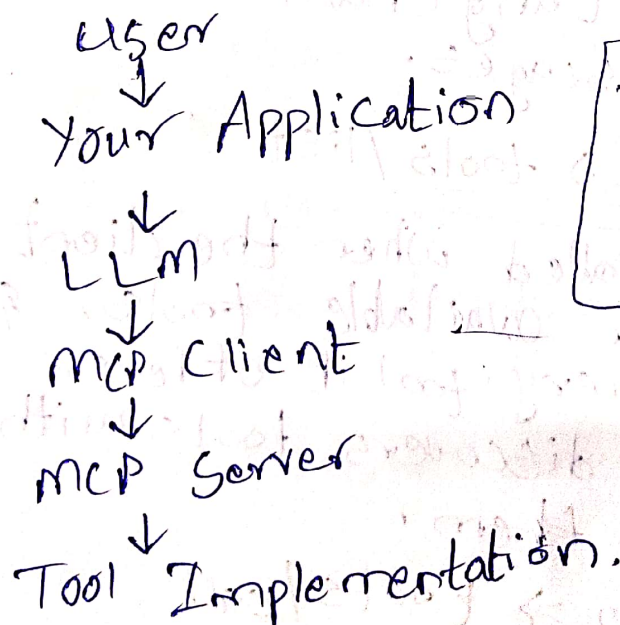
2) Streamable HTTP Transport



* Best for remote / cloud deployment.

* Multiple clients can connect.

Agent Architecture ⇒ After MCP



* MCP inserts a standardized execution tool.

* MCP does not change the working of agent loop.

After MCP

LLM asks for multiply Tool

↓
Agent (APP) creates MCP client

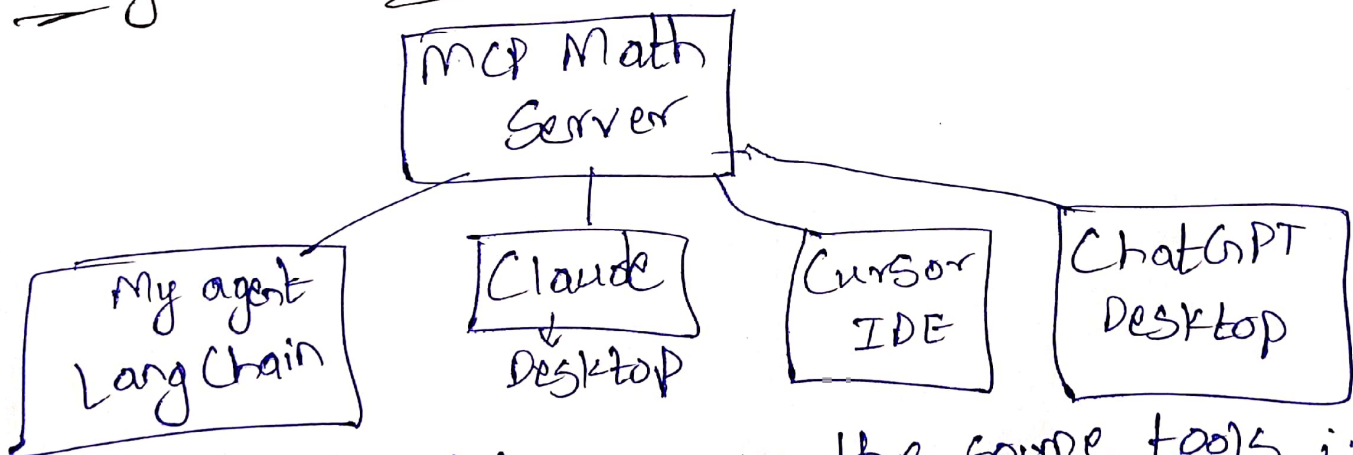
↓
MCP client → MCP Server

↓
MCP server executes tool

↓
MCP server returns result.

↓
Agent (APP) sends result to LLM.

Why MCP makes your Agent better



* All the 4 apps can use the same tools i.e; build once & use everywhere.

- Tools can be written in one server file, so all AI apps discover them automatically.
- If a bug is fixed in a tool → all the connected agents can get the fix instantly.
- Agent code stays clean.
- Mix & matches servers.